

Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

# Звіт до комп’ютерного практикуму №2

## з дисципліни «Комп’ютерне моделювання дискретно-подійних систем»

Прийняв:
Викладач
Д.С. Коринецький

Виконав
Студент 4 курсу, гр. ІП-33
Ломакін Максим

Оцінка: _____

«___» _____ 2026 р.

Київ 2026

**Об’єктно-орієнтований підхід до побудови імітаційних моделей дискретно-подійних систем**

## 1. Текст завдання:

1. Реалізувати алгоритм імітації простої моделі обслуговування одним пристроєм з використанням об’єктно-орієнтованого підходу. 5 балів.
2. Модифікувати алгоритм, додавши обчислення середнього завантаження пристрою. 5 балів.
3. Створити модель за схемою, представленою на рисунку 1. 30 балів.
4. Виконати верифікацію моделі, змінюючи значення вхідних змінних та параметрів моделі. Навести результати верифікації у таблиці. 10 балів.
5. Модифікувати клас PROCESS, щоб можна було його використовувати для моделювання процесу обслуговування кількома ідентичними пристроями. 20 балів.
6. Модифікувати клас PROCESS, щоб можна було організовувати вихід в два і більше наступних блоків, в тому числі з поверненням у попередні блоки. 30 балів.

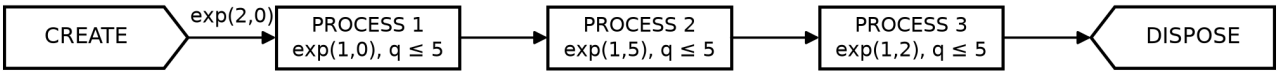



Рисунок 1 – Схема моделі (параметри – базовий варіант, використаний у роботі)

## 2. Код програми:

Програму написано мовою Python 3.12 за структурою класів із методичних вказівок (Element, Create, Process, Model, FunRand). Вихідний код і результати розміщено в репозиторії https://github.com/kqjqxb/Computer-simulation (каталог `lab2`). Призначення файлів:

- `fun_rand.py` – генератори випадкових затримок (експоненційний, рівномірний, нормальний, Ерланга);
- `element.py` – базовий клас Element: час наступної події, затримка, маршрути виходу;
- `elements.py` – класи Create, Process (черга, кілька каналів, статистика) та Dispose;

- `model.py` – клас Model: просування часу за принципом найближчої події та збирання результатів;
- `queueing_theory.py` – аналітичні формули систем M/M/c/K і мереж Джексона для верифікації;
- `runner.py` – серії прогонів з різними зернами генератора та довірчі інтервали;
- `task1_simple.py`, `task3_chain.py`, `task4_verification.py`, `task5_multichannel.py`, `task6_routing.py` – моделі та експерименти для пунктів завдання.

Файл fun_rand.py

```
import math
import random

_rng = random.Random()


def seed(value):
    _rng.seed(value)


def exp(time_mean):
    a = 0.0
    while a == 0.0:
        a = _rng.random()
    return -time_mean * math.log(a)


def unif(time_min, time_max):
    a = 0.0
    while a == 0.0:
        a = _rng.random()
    return time_min + a * (time_max - time_min)


def norm(time_mean, time_deviation):
    return _rng.gauss(time_mean, time_deviation)


def erlang(time_mean, k):
    product = 1.0
    for _ in range(k):
        product *= 1.0 - _rng.random()
    return -time_mean / k * math.log(product)


def uniform01():
    return _rng.random()
```

Файл element.py

```
import math

import fun_rand


class Element:
    _next_id = 0

    def __init__(self, name=None, delay_mean=1.0, delay_dev=0.0, distribution="exp"):
        self.id = Element._next_id
        Element._next_id += 1
        self.name = name or f"element{self.id}"
        self.delay_mean = delay_mean
        self.delay_dev = delay_dev
```

```
        self.distribution = distribution
        self.tnext = math.inf
        self.tcurr = 0.0
        self.state = 0
        self.quantity = 0
        self.routes = []
        self.route_counts = []

    def set_next_element(self, element):
        self.routes = [(element, 1.0)]
        self.route_counts = [0]

    def add_route(self, element, probability):
        """Додає ще один вихід з елемента; сума ймовірностей усіх виходів має дорівнювати 1."""
        self.routes.append((element, probability))
        self.route_counts.append(0)

    def next_element(self):
        if not self.routes:
            return None
        index = 0
        if len(self.routes) > 1:
            z = fun_rand.uniform01()
            cumulative = 0.0
            for index, (_, probability) in enumerate(self.routes):
                cumulative += probability
                if z < cumulative:
                    break
        self.route_counts[index] += 1
        return self.routes[index][0]

    def get_delay(self):
        kind = self.distribution.lower()
        if kind == "exp":
            return fun_rand.exp(self.delay_mean)
        if kind == "norm":
            return max(0.0, fun_rand.norm(self.delay_mean, self.delay_dev))
        if kind == "unif":
            return fun_rand.unif(self.delay_mean - self.delay_dev, self.delay_mean + self.delay_dev)
        if kind == "erlang":
            return fun_rand.erlang(self.delay_mean, int(self.delay_dev))
        return self.delay_mean

    def in_act(self):
        pass

    def out_act(self):
        self.quantity += 1

    def do_statistics(self, delta):
        pass

    def print_info(self):
        print(f"{self.name} state = {self.state} quantity = {self.quantity} tnext = {self.tnext:.4f}")

    def print_result(self):
        print(f"{self.name} quantity = {self.quantity}")
```

## Файл elements.py

```
import math

from element import Element


class Create(Element):
    def __init__(self, delay_mean, **kwargs):
        super().__init__(delay_mean=delay_mean, **kwargs)
        self.tnext = 0.0  # імітація розпочнеться з події Create

    def out_act(self):
        super().out_act()
        self.tnext = self.tcurr + self.get_delay()
        target = self.next_element()
        if target is not None:
            target.in_act()
```

```
class Dispose(Element):
    def __init__(self, name="DISPOSE"):
        super().__init__(name=name)

    def in_act(self):
        self.quantity += 1


class Process(Element):
    """Пристрій обслуговування з чергою обмеженої довжини та кількома ідентичними каналами."""

    def __init__(self, delay_mean, channels=1, maxqueue=math.inf, **kwargs):
        super().__init__(delay_mean=delay_mean, **kwargs)
        self.channels = channels
        self.channel_tnext = [math.inf] * channels
        self.maxqueue = maxqueue
        self.queue = 0
        self.failure = 0
        self.arrivals = 0
        self.queue_area = 0.0
        self.busy_area = 0.0

    @property
    def busy(self):
        return sum(t != math.inf for t in self.channel_tnext)

    def _refresh(self):
        self.tnext = min(self.channel_tnext)
        self.state = self.busy

    def in_act(self):
        self.arrivals += 1
        if self.busy < self.channels:
            free = self.channel_tnext.index(math.inf)
            self.channel_tnext[free] = self.tcurr + self.get_delay()
        elif self.queue < self.maxqueue:
            self.queue += 1
        else:
            self.failure += 1
        self._refresh()

    def out_act(self):
        finished = [i for i, t in enumerate(self.channel_tnext) if t == self.tcurr]
        for i in finished:
            self.quantity += 1
            self.channel_tnext[i] = math.inf
            if self.queue > 0:
                self.queue -= 1
                self.channel_tnext[i] = self.tcurr + self.get_delay()
        self._refresh()
        for _ in finished:
            target = self.next_element()
            if target is not None:
                target.in_act()

    def do_statistics(self, delta):
        self.queue_area += self.queue * delta
        self.busy_area += self.busy * delta

    def mean_queue(self, total_time):
        return self.queue_area / total_time

    def mean_load(self, total_time):
        """Середнє завантаження: частка часу, протягом якої канали пристрою зайняті."""
        return self.busy_area / (total_time * self.channels)

    def failure_probability(self):
        return self.failure / self.arrivals if self.arrivals else 0.0

    def print_info(self):
        print(f"{self.name} state = {self.state} queue = {self.queue} quantity = {self.quantity} "
              f"failure = {self.failure} tnext = {self.tnext:.4f}")
```

Файл model.py

```python
import math

from elements import Create, Dispose, Process


class Model:
    def __init__(self, elements):
        self.elements = elements
        self.tcurr = 0.0
        self.in_system_area = 0.0

    def in_system(self):
        return sum(e.queue + e.busy for e in self.elements if isinstance(e, Process))

    def simulate(self, time, trace_events=0):
        events = 0
        while self.tcurr < time:
            tnext = min(e.tnext for e in self.elements)
            delta = min(tnext, time) - self.tcurr
            for e in self.elements:
                e.do_statistics(delta)
            self.in_system_area += self.in_system() * delta
            if tnext > time:
                self.tcurr = time
                break

            self.tcurr = tnext
            for e in self.elements:
                e.tcurr = self.tcurr
            active = [e for e in self.elements if e.tnext == self.tcurr]
            for e in active:
                e.out_act()

            events += 1
            if events <= trace_events:
                names = ", ".join(e.name for e in active)
                print(f"\nIt's time for event in {names}, time = {self.tcurr:.4f}")
                for e in self.elements:
                    e.print_info()
        for e in self.elements:
            e.tcurr = self.tcurr

    def results(self):
        total = self.tcurr
        created = sum(e.quantity for e in self.elements if isinstance(e, Create))
        disposed = sum(e.quantity for e in self.elements if isinstance(e, Dispose))
        failures = sum(e.failure for e in self.elements if isinstance(e, Process))
        mean_in_system = self.in_system_area / total
        throughput = disposed / total
        out = {
            "time": total, "created": created, "disposed": disposed, "failures": failures,
            "in_system_now": self.in_system(),
            "failure_probability": failures / created if created else 0.0,
            "mean_in_system": mean_in_system,
            # формула Літтла: середній час перебування = середня кількість у системі / інтенсивність виходу
            "mean_time_in_system": mean_in_system / throughput if throughput else math.nan,
            "processes": {},
        }
        for e in self.elements:
            if isinstance(e, Process):
                out["processes"][e.name] = {
                    "arrivals": e.arrivals, "served": e.quantity, "failure": e.failure,
                    "failure_probability": e.failure_probability(),
                    "mean_queue": e.mean_queue(total), "mean_load": e.mean_load(total),
                }
        return out

    def print_result(self):
        r = self.results()
        print("\n-------------RESULTS-------------")
        for e in self.elements:
            e.print_result()
            if isinstance(e, Process):
                p = r["processes"][e.name]
                print(f"    failure = {e.failure}, failure probability = {p['failure_probability']:.4f}")
                print(f"    mean length of queue = {p['mean_queue']:.4f}")
                print(f"    mean load = {p['mean_load']:.4f}")
```

```
        print(f"created = {r['created']}, disposed = {r['disposed']}, failures = {r['failures']}, "
              f"in system at the end = {r['in_system_now']}")
        print(f"failure probability = {r['failure_probability']:.4f}, "
              f"mean number in system = {r['mean_in_system']:.4f}, "
              f"mean time in system = {r['mean_time_in_system']:.4f}")
        return r
```

## Файл queueing_theory.py

```
"""Аналітичні формули теорії масового обслуговування для верифікації імітаційної моделі."""
import math


def mmck(arrival_rate, service_mean, channels=1, capacity_queue=math.inf):
    """Система M/M/c/K: c каналів, черга до capacity_queue місць (K = c + capacity_queue)."""
    a = arrival_rate * service_mean
    c = channels
    if capacity_queue == math.inf:
        rho = a / c
        if rho >= 1:
            raise ValueError("система нестаціонарна: ρ ≥ 1")
        p0 = 1 / (sum(a ** n / math.factorial(n) for n in range(c))
                  + a ** c / (math.factorial(c) * (1 - rho)))
        lq = p0 * a ** c * rho / (math.factorial(c) * (1 - rho) ** 2)
        return {"failure_probability": 0.0, "mean_queue": lq, "mean_load": rho,
                "mean_in_system": lq + a, "throughput": arrival_rate}

    k = c + int(capacity_queue)
    weights = [a ** n / math.factorial(n) if n <= c else a ** n / (math.factorial(c) * c ** (n - c))
               for n in range(k + 1)]
    total = sum(weights)
    p = [w / total for w in weights]
    p_fail = p[k]
    lq = sum((n - c) * p[n] for n in range(c + 1, k + 1))
    busy = sum(min(n, c) * p[n] for n in range(k + 1))
    return {"failure_probability": p_fail, "mean_queue": lq, "mean_load": busy / c,
            "mean_in_system": lq + busy, "throughput": arrival_rate * (1 - p_fail)}


def jackson_rates(external, routing):
    """Розв'язує рівняння балансу потоків λ_j = λ0_j + Σ_i λ_i · r_ij методом простої ітерації."""
    n = len(external)
    rates = list(external)
    for _ in range(10_000):
        new = [external[j] + sum(rates[i] * routing[i][j] for i in range(n)) for j in range(n)]
        if max(abs(x - y) for x, y in zip(new, rates)) < 1e-13:
            return new
        rates = new
    return rates
```

## Файл runner.py

```
import statistics

import fun_rand

# квантилі t-розподілу Стьюдента рівня 0.975 за кількістю ступенів вільності
T_95 = {1: 12.706, 2: 4.303, 3: 3.182, 4: 2.776, 5: 2.571, 6: 2.447, 7: 2.365, 8: 2.306, 9: 2.262, 19: 2.093}


def replicate(build_model, time, seeds):
    """Запускає модель з кількома зернами генератора; повертає список результатів model.results()."""
    runs = []
    for s in seeds:
        fun_rand.seed(s)
        model = build_model()
        model.simulate(time)
        runs.append(model.results())
    return runs


def summary(values):
    """Середнє та півширина 95 % довірчого інтервалу за t-розподілом Стьюдента."""
    mean = statistics.mean(values)
    if len(values) < 2:
        return mean, 0.0
```

```
    return mean, T_95.get(len(values) - 1, 1.96) * statistics.stdev(values) / len(values) ** 0.5


def metric(runs, path):
    values = []
    for r in runs:
        value = r
        for key in path:
            value = value[key]
        values.append(value)
    return summary(values)
```

## Файл task1_simple.py

```
"""Завдання 1-2: модель обслуговування одним пристроєм та середнє завантаження пристрою."""
import json
from pathlib import Path

import fun_rand
from elements import Create, Dispose, Process
from model import Model
from queueing_theory import mmck
from runner import metric, replicate

RESULTS = Path(__file__).resolve().parent / "results"
CREATE_MEAN, PROCESS_MEAN, MAXQUEUE = 2.0, 1.0, 5


def build():
    create = Create(CREATE_MEAN, name="CREATOR")
    process = Process(PROCESS_MEAN, maxqueue=MAXQUEUE, name="PROCESSOR")
    dispose = Dispose()
    create.set_next_element(process)
    process.set_next_element(dispose)
    return Model([create, process, dispose])


def main():
    fun_rand.seed(1)
    model = build()
    print(f"Create: exp({CREATE_MEAN}), Process: exp({PROCESS_MEAN}), maxqueue = {MAXQUEUE}, "
          "час моделювання 1000")
    model.simulate(1000.0, trace_events=6)
    single = model.print_result()

    theory = mmck(1 / CREATE_MEAN, PROCESS_MEAN, 1, MAXQUEUE)
    runs = replicate(build, 100_000.0, range(1, 11))
    print("\nПорівняння з теорією M/M/1/5 (10 прогонів по 100000 одиниць часу, 95 % довірчий інтервал):")
    rows = []
    for label, key in [("ймовірність відмови", "failure_probability"),
                       ("середня довжина черги", "mean_queue"),
                       ("середнє завантаження", "mean_load")]:
        mean, half = metric(runs, ["processes", "PROCESSOR", key])
        rows.append({"metric": label, "sim": mean, "ci": half, "theory": theory[key]})
        print(f"{label:24} імітація = {mean:.4f} ± {half:.4f}, теорія = {theory[key]:.4f}")

    with open(RESULTS / "task1.json", "w", encoding="utf-8") as f:
        json.dump({"single": single, "comparison": rows}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

## Файл task3_chain.py

```
"""Завдання 3: модель CREATE -> PROCESS 1 -> PROCESS 2 -> PROCESS 3 -> DISPOSE (рисунок 2.1)."""
import json
import math
from pathlib import Path

import fun_rand
from elements import Create, Dispose, Process
from model import Model

RESULTS = Path(__file__).resolve().parent / "results"
```

```
BASE = {
    "create_mean": 2.0, "create_dist": "exp",
    "means": (1.0, 1.5, 1.2), "dists": ("exp", "exp", "exp"), "devs": (0.0, 0.0, 0.0),
    "maxqueues": (5, 5, 5), "channels": (1, 1, 1),
}


def build(params=None):
    p = {**BASE, **(params or {})}
    create = Create(p["create_mean"], distribution=p["create_dist"], name="CREATE")
    settings = zip(p["means"], p["dists"], p["devs"], p["maxqueues"], p["channels"])
    processes = [Process(mean, channels=ch, maxqueue=q, distribution=dist, delay_dev=dev,
                         name=f"PROCESS {i + 1}")
                 for i, (mean, dist, dev, q, ch) in enumerate(settings)]
    dispose = Dispose()
    create.set_next_element(processes[0])
    for current, following in zip(processes, processes[1:] + [dispose]):
        current.set_next_element(following)
    return Model([create, *processes, dispose])


def main():
    fun_rand.seed(3)
    model = build()
    print("Create: exp(2.0); PROCESS 1-3: exp(1.0), exp(1.5), exp(1.2); maxqueue = 5; час моделювання 1000")
    model.simulate(1000.0, trace_events=4)
    result = model.print_result()
    balance = result["created"] - result["disposed"] - result["failures"] - result["in_system_now"]
    print(f"Баланс вимог: створено - вийшло - відмови - у системі = {balance}")
    with open(RESULTS / "task3.json", "w", encoding="utf-8") as f:
        json.dump(result, f, ensure_ascii=False, indent=1, default=lambda v: None if math.isinf(v) else v)


if __name__ == "__main__":
    main()
```

## Файл task4_verification.py

```
"""Завдання 4: верифікація моделі з рисунка 2.1 зміною вхідних змінних і параметрів."""
import json
import math
from pathlib import Path

import fun_rand
from queueing_theory import mmck
from runner import metric, replicate
from task3_chain import build

RESULTS = Path(__file__).resolve().parent / "results"
TIME = 10_000.0
INF = math.inf

EXPERIMENTS = [
    ("базові параметри", {}),
    ("Create: exp(1.0)", {"create_mean": 1.0}),
    ("Create: exp(4.0)", {"create_mean": 4.0}),
    ("PROCESS 2: exp(3.0)", {"means": (1.0, 3.0, 1.2)}),
    ("maxqueue = 0", {"maxqueues": (0, 0, 0)}),
    ("maxqueue = 1", {"maxqueues": (1, 1, 1)}),
    ("maxqueue = ∞", {"maxqueues": (INF, INF, INF)}),
    ("усі затримки сталі", {"create_dist": "", "dists": ("", "", "")}),
    ("PROCESS: unif(mean ± 0.5)", {"dists": ("unif",) * 3, "devs": (0.5,) * 3}),
    ("PROCESS: norm(mean; 0.3)", {"dists": ("norm",) * 3, "devs": (0.3,) * 3}),
    ("PROCESS: Ерланга k = 4", {"dists": ("erlang",) * 3, "devs": (4,) * 3}),
]


def variation_table():
    rows = []
    print(f"ВЕРИФІКАЦІЯ: зміна вхідних змінних і параметрів (час моделювання {TIME:.0f}, однакове зерно)")
    header = (f"{'№':>2} {'зміна':26} {'створ.':>6} {'вийшло':>6} {'відмови P1/P2/P3':>17} "
              f"{'черга P1/P2/P3':>17} {'завант. P1/P2/P3':>17} {'Рвідм':>6} {'Тсист':>7} {'баланс':>6}")
    print(header)
    for i, (label, params) in enumerate(EXPERIMENTS, 1):
        fun_rand.seed(4)
        model = build(params)
        model.simulate(TIME)
```

```
        r = model.results()
        p = list(r["processes"].values())
        balance = r["created"] - r["disposed"] - r["failures"] - r["in_system_now"]
        rows.append({"label": label, "created": r["created"], "disposed": r["disposed"],
                     "failures": [x["failure"] for x in p], "queues": [x["mean_queue"] for x in p],
                     "loads": [x["mean_load"] for x in p], "p_fail": r["failure_probability"],
                     "t_system": r["mean_time_in_system"], "balance": balance})
        print(f"{i:>2} {label:26} {r['created']:>6} {r['disposed']:>6} "
              f"{'/'.join(str(x['failure']) for x in p):>17} "
              f"{'/'.join(f'{x['mean_queue']:.2f}' for x in p):>17} "
              f"{'/'.join(f'{x['mean_load']:.3f}' for x in p):>17} "
              f"{r['failure_probability']:>6.3f} {r['mean_time_in_system']:>7.3f} {balance:>6}")
    return rows


def theory_table():
    seeds = range(1, 11)
    out = []
    print("\nПОРІВНЯННЯ З ТЕОРІЄЮ (10 прогонів по 20000 одиниць часу, 95 % довірчий інтервал)")

    print("\n1) Необмежені черги: кожен пристрій – система M/M/1 з λ = 0.5 (теорема Берка)")
    runs = replicate(lambda: build({"maxqueues": (INF, INF, INF)}), 20_000.0, seeds)
    t_total = 0.0
    for name, mean in zip(["PROCESS 1", "PROCESS 2", "PROCESS 3"], [1.0, 1.5, 1.2]):
        th = mmck(0.5, mean)
        t_total += th["mean_in_system"] / 0.5
        for label, key in [("завантаження", "mean_load"), ("середня черга", "mean_queue")]:
            sim, ci = metric(runs, ["processes", name, key])
            out.append({"case": "M/M/1", "element": name, "metric": label, "sim": sim, "ci": ci,
                        "theory": th[key]})
            print(f"{name:10} {label:14} імітація = {sim:.4f} ± {ci:.4f}, теорія = {th[key]:.4f}")
    sim, ci = metric(runs, ["mean_time_in_system"])
    out.append({"case": "M/M/1", "element": "модель", "metric": "час перебування", "sim": sim, "ci": ci,
                "theory": t_total})
    print(f"{'модель':10} {'час у системі':14} імітація = {sim:.4f} ± {ci:.4f}, теорія = {t_total:.4f}")

    print("\n2) PROCESS 1 з обмеженою чергою: система M/M/1/K, λ = 0.5, середній час обслуговування 1.0")
    for q in (0, 1, 2, 5):
        runs = replicate(lambda q=q: build({"maxqueues": (q, INF, INF)}), 20_000.0, seeds)
        th = mmck(0.5, 1.0, 1, q)
        for label, key in [("ймовірність відмови", "failure_probability"), ("завантаження", "mean_load"),
                           ("середня черга", "mean_queue")]:
            sim, ci = metric(runs, ["processes", "PROCESS 1", key])
            out.append({"case": f"M/M/1/{q + 1}", "element": "PROCESS 1", "metric": label,
                        "sim": sim, "ci": ci, "theory": th[key]})
            print(f"maxqueue = {q}: {label:20} імітація = {sim:.4f} ± {ci:.4f}, теорія = {th[key]:.4f}")
    return out


def main():
    rows = variation_table()
    theory = theory_table()
    with open(RESULTS / "task4.json", "w", encoding="utf-8") as f:
        json.dump({"variations": rows, "theory": theory}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

## Файл task5_multichannel.py

```
"""Завдання 5: PROCESS з кількома ідентичними каналами обслуговування."""
import json
import math
from pathlib import Path

import fun_rand
from elements import Create, Dispose, Process
from model import Model
from queueing_theory import mmck
from runner import metric, replicate

RESULTS = Path(__file__).resolve().parent / "results"
CREATE_MEAN, PROCESS_MEAN = 1.0, 2.5


def build(channels, maxqueue):
```

```
    create = Create(CREATE_MEAN, name="CREATE")
    process = Process(PROCESS_MEAN, channels=channels, maxqueue=maxqueue, name="PROCESS")
    dispose = Dispose()
    create.set_next_element(process)
    process.set_next_element(dispose)
    return Model([create, process, dispose])


def main():
    fun_rand.seed(5)
    model = build(3, 3)
    print(f"Create: exp({CREATE_MEAN}), PROCESS: exp({PROCESS_MEAN}), 3 канали, maxqueue = 3")
    model.simulate(1000.0, trace_events=6)
    model.print_result()

    rows = []
    print(f"\nПОРІВНЯННЯ З ТЕОРІЄЮ M/M/c/K (λ = {1 / CREATE_MEAN}, "
          f"середній час обслуговування {PROCESS_MEAN}; 20 прогонів по 50000 одиниць часу)")
    for channels, maxqueue in [(1, 3), (2, 3), (3, 3), (4, 3), (5, 3), (3, math.inf), (4, math.inf)]:
        runs = replicate(lambda: build(channels, maxqueue), 50_000.0, range(1, 21))
        th = mmck(1 / CREATE_MEAN, PROCESS_MEAN, channels, maxqueue)
        row = {"channels": channels, "maxqueue": None if maxqueue == math.inf else maxqueue}
        line = [f"c = {channels}, maxqueue = {maxqueue}:"]
        for label, key in [("Рвідм", "failure_probability"), ("завант.", "mean_load"),
                           ("черга", "mean_queue")]:
            sim, ci = metric(runs, ["processes", "PROCESS", key])
            row[key] = {"sim": sim, "ci": ci, "theory": th[key]}
            line.append(f"{label} {sim:.4f} ± {ci:.4f} (теорія {th[key]:.4f})")
        rows.append(row)
        print("  ".join(line))

    with open(RESULTS / "task5.json", "w", encoding="utf-8") as f:
        json.dump(rows, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

## Файл task6_routing.py

```
"""Завдання 6: вихід з PROCESS у два і більше наступних блоків, зокрема з поверненням у попередні блоки."""
import json
import math
from pathlib import Path

import fun_rand
from elements import Create, Dispose, Process
from model import Model
from queueing_theory import jackson_rates, mmck
from runner import metric, replicate

RESULTS = Path(__file__).resolve().parent / "results"
CREATE_MEAN = 2.0
NODES = [("PROCESS 1", 1.0, 1), ("PROCESS 2", 1.6, 2), ("PROCESS 3", 1.2, 1)]
# ймовірності переходів між пристроями; решта ймовірності з PROCESS 3 – вихід з моделі
ROUTING = [
    [0.0, 1.0, 0.0],
    [0.3, 0.0, 0.7],
    [0.0, 0.2, 0.0],
]
EXIT_P3 = 0.8


def build(maxqueue=math.inf):
    create = Create(CREATE_MEAN, name="CREATE")
    p1, p2, p3 = [Process(mean, channels=ch, maxqueue=maxqueue, name=name) for name, mean, ch in NODES]
    dispose = Dispose()
    create.set_next_element(p1)
    p1.set_next_element(p2)
    p2.add_route(p3, ROUTING[1][2])
    p2.add_route(p1, ROUTING[1][0])
    p3.add_route(dispose, EXIT_P3)
    p3.add_route(p2, ROUTING[2][1])
    return Model([create, p1, p2, p3, dispose])


def main():
```

```
    fun_rand.seed(6)
    model = build()
    print("CREATE exp(2.0) -> PROCESS 1 exp(1.0) -> PROCESS 2 exp(1.6), 2 канали -> "
          "{PROCESS 3: 0.7; PROCESS 1: 0.3}; PROCESS 3 exp(1.2) -> {DISPOSE: 0.8; PROCESS 2: 0.2}")
    model.simulate(1000.0, trace_events=5)
    single = model.print_result()
    elements = {e.name: e for e in model.elements}
    print("Фактичні частки переходів за маршрутами:")
    for name in ("PROCESS 2", "PROCESS 3"):
        e = elements[name]
        total = sum(e.route_counts)
        shares = ", ".join(f"{target.name}: {count / total:.3f} (задано {prob})"
                           for (target, prob), count in zip(e.routes, e.route_counts))
        print(f"  {name}: {shares}")

    rates = jackson_rates([1 / CREATE_MEAN, 0.0, 0.0], ROUTING)
    print(f"\nРівняння балансу потоків: λ1 = {rates[0]:.4f}, λ2 = {rates[1]:.4f}, λ3 = {rates[2]:.4f}")
    runs = replicate(build, 50_000.0, range(1, 21))
    rows = []
    total_l = 0.0
    print("ПОРІВНЯННЯ З ТЕОРІЄЮ МЕРЕЖ ДЖЕКСОНА (необмежені черги, 20 прогонів по 50000 одиниць часу)")
    for (name, mean, channels), rate in zip(NODES, rates):
        th = mmck(rate, mean, channels)
        th["visit_ratio"] = rate * CREATE_MEAN
        total_l += th["mean_in_system"]
        visits = [r["processes"][name]["arrivals"] / r["created"] for r in runs]
        sim_v = sum(visits) / len(visits)
        for label, key in [("завантаження", "mean_load"), ("середня черга", "mean_queue")]:
            sim, ci = metric(runs, ["processes", name, key])
            rows.append({"element": name, "metric": label, "sim": sim, "ci": ci, "theory": th[key]})
            print(f"{name:10} {label:22} імітація = {sim:.4f} ± {ci:.4f}, теорія = {th[key]:.4f}")
        rows.append({"element": name, "metric": "надходжень на одну вимогу", "sim": sim_v, "ci": None,
                     "theory": th["visit_ratio"]})
        print(f"{name:10} {'надходжень на 1 вимогу':22} імітація = {sim_v:.4f}, "
              f"теорія = {th['visit_ratio']:.4f}")
    t_theory = total_l * CREATE_MEAN
    sim, ci = metric(runs, ["mean_time_in_system"])
    rows.append({"element": "модель", "metric": "час перебування", "sim": sim, "ci": ci, "theory": t_theory})
    print(f"{'модель':10} {'час перебування':22} імітація = {sim:.4f} ± {ci:.4f}, теорія = {t_theory:.4f}")

    print("\nТА САМА МЕРЕЖА З ОБМЕЖЕНИМИ ЧЕРГАМИ (maxqueue = 2), 10 прогонів по 20000 одиниць часу:")
    limited = replicate(lambda: build(2), 20_000.0, range(1, 11))
    finite = {}
    for name, _, _ in NODES:
        values = {key: metric(limited, ["processes", name, key])[0]
                  for key in ("failure_probability", "mean_queue", "mean_load")}
        finite[name] = values
        print(f"{name:10} Рвідм = {values['failure_probability']:.4f}, черга = {values['mean_queue']:.4f}, "
              f"завантаження = {values['mean_load']:.4f}")
    p_fail, _ = metric(limited, ["failure_probability"])
    t_sys, _ = metric(limited, ["mean_time_in_system"])
    print(f"частка втрачених вимог = {p_fail:.4f}, середній час перебування = {t_sys:.4f}")
    finite["model"] = {"failure_probability": p_fail, "mean_time_in_system": t_sys}

    shares = {name: [c / sum(elements[name].route_counts) for c in elements[name].route_counts]
              for name in ("PROCESS 2", "PROCESS 3")}
    with open(RESULTS / "task6.json", "w", encoding="utf-8") as f:
        json.dump({"single": single, "rates": rates, "theory": rows, "finite": finite, "route_shares":
shares},
                  f, ensure_ascii=False, indent=1, default=lambda v: None)


if __name__ == "__main__":
    main()
```

## 3. Результати роботи програми:

Завдання 1:

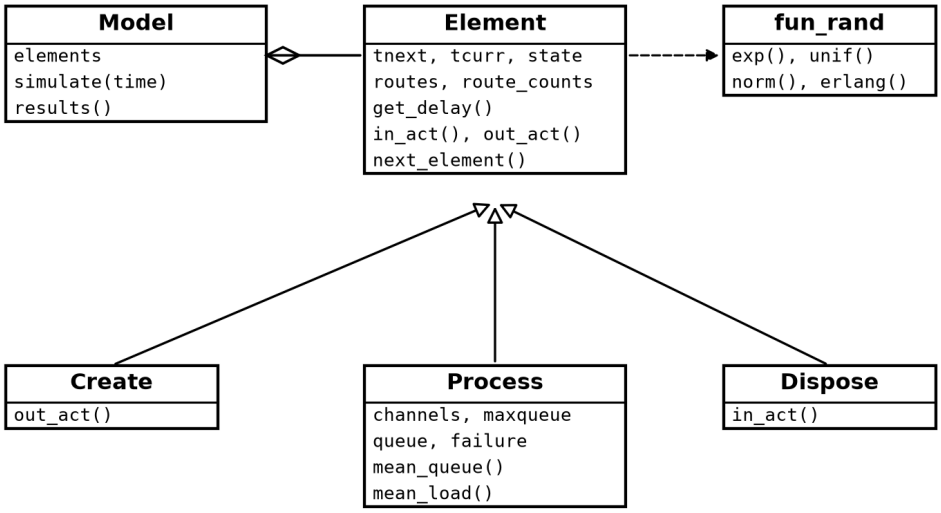



Рисунок 2 – Діаграма класів програми

Структура програми (рис. 2) повторює діаграму класів із методичних вказівок. Усі елементи моделі є нащадками класу Element, який зберігає момент наступної події `tnext`, поточний час `tcurr`, параметри затримки та маршрут вимоги до наступного елемента. Create генерує вимоги, Process обслуговує їх із чергою обмеженої довжини, Dispose фіксує вихід вимог з моделі. Метод `Model.simulate` реалізує принцип найближчої події: знаходить найменший `tnext` серед усіх елементів, накопичує статистику за інтервал до цієї події, просуває модельний час і викликає `out_act()` усіх елементів, у яких подія припадає на поточний момент.

На відміну від лістингу методички, елемент наступної події визначається безпосередньо, а не через індекс у списку, що збігається з id лише за певного порядку створення об’єктів, а модельний час не виходить за межу моделювання: статистика за останній інтервал накопичується до моменту закінчення моделювання.

Проста модель CREATE → PROCESS → DISPOSE з параметрами з методички: інтервал надходження exp(2,0), час обслуговування exp(1,0), максимальна довжина черги 5, час моделювання 1000. Фрагмент виводу програми (трасування перших подій і результати):

```
Create: exp(2.0), Process: exp(1.0), maxqueue = 5, час моделювання 1000

It's time for event in CREATOR, time = 0.0000
CREATOR state = 0 quantity = 1 tnext = 4.0144
PROCESSOR state = 1 queue = 0 quantity = 0 failure = 0 tnext = 0.1655
DISPOSE state = 0 quantity = 0 tnext = inf

It's time for event in PROCESSOR, time = 0.1655
CREATOR state = 0 quantity = 1 tnext = 4.0144
PROCESSOR state = 0 queue = 0 quantity = 1 failure = 0 tnext = inf
DISPOSE state = 0 quantity = 1 tnext = inf

It's time for event in CREATOR, time = 4.0144
CREATOR state = 0 quantity = 2 tnext = 4.5534
PROCESSOR state = 1 queue = 0 quantity = 1 failure = 0 tnext = 5.3806
DISPOSE state = 0 quantity = 1 tnext = inf

...

-------------RESULTS-------------
CREATOR quantity = 519
PROCESSOR quantity = 515
    failure = 3, failure probability = 0.0058
    mean length of queue = 0.3884
    mean load = 0.5005
DISPOSE quantity = 515
created = 519, disposed = 515, failures = 3, in system at the end = 1
failure probability = 0.0058, mean number in system = 0.8889, mean time in system = 1.7260
```

За 1000 одиниць часу створено 519 вимог, обслуговано 515, відмовлено 3; середня довжина черги 0,388. Результати близькі до наведених у методичних вказівках (508 створених, 501 обслугована, 6 відмов).

Завдання 2:

Середнє завантаження пристрою – це частка часу, протягом якої пристрій зайнятий. Метод `do_statistics(delta)` після кожного інтервалу між подіями додає до накопичувача добуток кількості зайнятих каналів на тривалість інтервалу, а результат ділиться на час моделювання (для багатоканального пристрою – ще й на кількість каналів):

$$U = (1 / (c \cdot T)) \cdot \Sigma\, n_{\text{зайн}}(t_k) \cdot (t_{k+1} - t_k)$$

Для перевірки обчислень використано те, що проста модель з експоненційними затримками є системою M/M/1/K, для якої завантаження, ймовірність відмови й середню довжину черги можна знайти аналітично (K = maxqueue + 1 = 6, ρ = 1,0/2,0 = 0,5). Модель запущено 10 разів по 100000 одиниць часу з різними зернами генератора; наведено середнє та 95 % довірчий інтервал.

Таблиця 1

Проста модель і теорія M/M/1/5

| **Показник** | **Імітація** | **Теорія M/M/1/K** |
|---|---|---|
| ймовірність відмови | 0,0078 ± 0,0004 | 0,0079 |
| середня довжина черги | 0,4502 ± 0,0052 | 0,4488 |
| середнє завантаження | 0,4970 ± 0,0018 | 0,4961 |

Обчислене завантаження 0,4970 збігається з теоретичним $\rho(1 - P_{відм}) = 0{,}4961$ (табл. 1); теоретичні значення решти показників також лежать у довірчих інтервалах.

Завдання 3:

Модель за схемою рисунка 1 складається з елемента CREATE, трьох послідовних пристроїв PROCESS 1–3 і DISPOSE. Модель будується функцією `build` з файлу `task3_chain.py`, яка приймає словник параметрів – це використано під час верифікації. Базові параметри: надходження exp(2,0); обслуговування exp(1,0), exp(1,5) і exp(1,2); максимальна довжина кожної черги 5. Фрагмент виводу для 1000 одиниць часу:

```
Create: exp(2.0); PROCESS 1-3: exp(1.0), exp(1.5), exp(1.2); maxqueue = 5; час моделювання 1000

It's time for event in CREATE, time = 0.0000
CREATE state = 0 quantity = 1 tnext = 2.8713
PROCESS 1 state = 1 queue = 0 quantity = 0 failure = 0 tnext = 0.6084
PROCESS 2 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
PROCESS 3 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
DISPOSE state = 0 quantity = 0 tnext = inf

It's time for event in PROCESS 1, time = 0.6084
CREATE state = 0 quantity = 1 tnext = 2.8713
PROCESS 1 state = 0 queue = 0 quantity = 1 failure = 0 tnext = inf
PROCESS 2 state = 1 queue = 0 quantity = 0 failure = 0 tnext = 2.0999
PROCESS 3 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
DISPOSE state = 0 quantity = 0 tnext = inf

...

------------RESULTS-------------
CREATE quantity = 508
PROCESS 1 quantity = 507
    failure = 1, failure probability = 0.0020
    mean length of queue = 0.3978
    mean load = 0.5037
PROCESS 2 quantity = 479
    failure = 28, failure probability = 0.0552
    mean length of queue = 1.2404
    mean load = 0.7274
PROCESS 3 quantity = 468
    failure = 11, failure probability = 0.0230
    mean length of queue = 0.6812
    mean load = 0.5730
DISPOSE quantity = 468
created = 508, disposed = 468, failures = 40, in system at the end = 0
failure probability = 0.0787, mean number in system = 4.1235, mean time in system = 8.8108
Баланс вимог: створено - вийшло - відмови - у системі = 0
```

Найбільш завантаженим є PROCESS 2 (завантаження 0,727, найбільший середній час обслуговування 1,5), тому саме в ньому найдовша черга та найбільше відмов (28). Із 508 створених вимог 468 вийшли з моделі, 40 отримали відмову, решта на момент закінчення перебували в системі; баланс «створено − вийшло − відмови − у системі» дорівнює нулю, тобто модель не губить і не дублює вимоги.

Завдання 4:

Верифікація виконувалась двома способами. Спочатку вхідні змінні та параметри моделі змінювались по одному відносно базового варіанта, а результати перевірялись на логічну узгодженість (табл. 2). Усі експерименти мають тривалість 10000 одиниць часу та однакове зерно генератора.

Таблиця 2

Результати верифікації моделі зміною вхідних змінних і параметрів

| № | Зміна відносно базових параметрів | Створено | Вийшло | Відмови P1 / P2 / P3 | Сер. черга P1 / P2 / P3 | Завантаження P1 / P2 / P3 | $P_{відм}$ | $T_{сист}$ | Баланс |
|---|---|---|---|---|---|---|---|---|---|
| 1 | базові параметри | 4917 | 4604 | 29 / 219 / 57 | 0,39 / 1,15 / 0,59 | 0,479 / 0,703 / 0,551 | 0,062 | 8,40 | 0 |
| 2 | Create: exp(1.0) | 9962 | 6061 | 1377 / 2241 / 274 | 2,17 / 3,06 / 1,24 | 0,858 / 0,940 / 0,725 | 0,391 | 14,83 | 0 |
| 3 | Create: exp(4.0) | 2510 | 2502 | 0 / 6 / 2 | 0,08 / 0,21 / 0,12 | 0,255 / 0,372 / 0,295 | 0,003 | 5,35 | 0 |
| 4 | PROCESS 2: exp(3.0) | 5078 | 3241 | 48 / 1781 / 3 | 0,49 / 3,49 / 0,23 | 0,510 / 0,964 / 0,389 | 0,361 | 18,75 | 0 |
| 5 | maxqueue = 0 | 4966 | 2175 | 1630 / 843 / 317 | 0,00 / 0,00 / 0,00 | 0,323 / 0,373 / 0,267 | 0,562 | 4,43 | 0 |
| 6 | maxqueue = 1 | 4962 | 3277 | 681 / 711 / 292 | 0,15 / 0,19 / 0,11 | 0,428 / 0,521 / 0,399 | 0,339 | 5,50 | 0 |
| 7 | maxqueue = ∞ | 4961 | 4959 | 0 / 0 / 0 | 0,50 / 2,68 / 0,81 | 0,494 / 0,758 / 0,578 | 0,000 | 11,74 | 0 |
| 8 | усі затримки сталі | 5001 | 4999 | 0 / 0 / 0 | 0,00 / 0,00 / 0,00 | 0,500 / 0,750 / 0,600 | 0,000 | 3,70 | 0 |
| 9 | PROCESS: unif(mean ± 0.5) | 4954 | 4905 | 4 / 42 / 0 | 0,27 / 0,70 / 0,03 | 0,498 / 0,734 / 0,589 | 0,009 | 5,77 | 0 |
| 10 | PROCESS: norm(mean; 0.3) | 5049 | 5017 | 2 / 29 / 0 | 0,27 / 0,79 / 0,04 | 0,507 / 0,751 / 0,603 | 0,006 | 5,91 | 0 |
| 11 | PROCESS: Ерланга k = 4 | 5106 | 5003 | 11 / 90 / 0 | 0,33 / 0,96 / 0,24 | 0,509 / 0,748 / 0,606 | 0,020 | 6,81 | 0 |

Аналіз таблиці 2:

- зменшення середнього інтервалу надходження з 2 до 1 удвічі збільшує кількість створених вимог (9962 проти 4917), завантаження всіх пристроїв, черги й частку відмов (0,391); збільшення інтервалу до 4 дає протилежний ефект – відмов майже немає;
- збільшення часу обслуговування в PROCESS 2 до 3,0 робить його «вузьким місцем»: завантаження наближається до 1, відмови зосереджуються в PROCESS 2, а наступний PROCESS 3 отримує менше вимог і розвантажується;
- за maxqueue = 0 черги порожні, а відмов найбільше; зі збільшенням maxqueue відмови зменшуються, а за необмежених черг зникають повністю, натомість зростають черги та час перебування в системі;
- за сталих затримок черги не утворюються (наступна вимога надходить через 2,0, а найдовше обслуговування триває 1,5), завантаження точно дорівнюють 1,0/2 = 0,5, 1,5/2 = 0,75, 1,2/2 = 0,6, а час перебування – сумі часів обслуговування 3,7 (отримано 3,70);
- за менш мінливих законів обслуговування (рівномірний, нормальний, Ерланга) з тими самими середніми черги й відмови значно менші, ніж за

експоненційного, – вплив дисперсії часу обслуговування відповідає теорії; завантаження при цьому майже не змінюється, бо залежить лише від середніх значень;

- в усіх експериментах баланс вимог дорівнює нулю.

Далі результати порівнювались з аналітичними формулами теорії масового обслуговування. За необмежених черг і експоненційних затримок вихідний потік кожного пристрою M/M/1 також пуассонівський (теорема Берка), тому кожен пристрій моделі є системою M/M/1 з $\lambda = 0{,}5$: завантаження $\rho = \lambda \cdot \tau$, середня черга $\rho^2/(1-\rho)$, а середній час перебування в моделі – сума $\tau/(1-\rho)$ для трьох пристроїв. Для PROCESS 1 з обмеженою чергою використано формули M/M/1/K. Виконано 10 прогонів по 20000 одиниць часу.

Таблиця 3

Порівняння результатів моделі з аналітичними розв’язками

| **Система** | **Елемент** | **Показник** | **Імітація (95 % ДІ)** | **Теорія** |
|---|---|---|---|---|
| M/M/1 | PROCESS 1 | завантаження | 0,5003 ± 0,0050 | 0,5000 |
| M/M/1 | PROCESS 1 | середня черга | 0,5045 ± 0,0227 | 0,5000 |
| M/M/1 | PROCESS 2 | завантаження | 0,7530 ± 0,0078 | 0,7500 |
| M/M/1 | PROCESS 2 | середня черга | 2,2823 ± 0,1455 | 2,2500 |
| M/M/1 | PROCESS 3 | завантаження | 0,6039 ± 0,0062 | 0,6000 |
| M/M/1 | PROCESS 3 | середня черга | 0,9428 ± 0,0564 | 0,9000 |
| M/M/1 | модель | час перебування | 11,1662 ± 0,3846 | 11,0000 |
| M/M/1/1 | PROCESS 1 | ймовірність відмови | 0,3365 ± 0,0036 | 0,3333 |
| M/M/1/1 | PROCESS 1 | завантаження | 0,3341 ± 0,0025 | 0,3333 |
| M/M/1/1 | PROCESS 1 | середня черга | 0,0000 ± 0,0000 | 0,0000 |
| M/M/1/2 | PROCESS 1 | ймовірність відмови | 0,1426 ± 0,0036 | 0,1429 |
| M/M/1/2 | PROCESS 1 | завантаження | 0,4281 ± 0,0029 | 0,4286 |
| M/M/1/2 | PROCESS 1 | середня черга | 0,1414 ± 0,0023 | 0,1429 |
| M/M/1/3 | PROCESS 1 | ймовірність відмови | 0,0659 ± 0,0034 | 0,0667 |
| M/M/1/3 | PROCESS 1 | завантаження | 0,4679 ± 0,0034 | 0,4667 |
| M/M/1/3 | PROCESS 1 | середня черга | 0,2679 ± 0,0053 | 0,2667 |
| M/M/1/6 | PROCESS 1 | ймовірність відмови | 0,0073 ± 0,0010 | 0,0079 |
| M/M/1/6 | PROCESS 1 | завантаження | 0,4956 ± 0,0051 | 0,4961 |
| M/M/1/6 | PROCESS 1 | середня черга | 0,4454 ± 0,0159 | 0,4488 |

Усі теоретичні значення в таблиці 3 лежать у довірчих інтервалах або на відстані, меншій за їх півширину, отже модель відтворює функціонування системи правильно.

Завдання 5:

Для обслуговування кількома ідентичними пристроями клас Process замість одного моменту звільнення зберігає список `channel_tnext` – момент звільнення кожного каналу (`inf` для вільного). Нова вимога займає перший вільний канал; якщо всі канали зайняті, вона стає в чергу або отримує відмову. Момент наступної події елемента дорівнює найменшому з моментів звільнення каналів, а в `out_act()` обробляються всі канали, що звільнилися в поточний момент: кожен із них бере вимогу з черги, а обслужені вимоги передаються далі. Стан елемента `state` дорівнює кількості зайнятих каналів. Кількість каналів задається параметром `channels` (за замовчуванням 1, тому попередні моделі не змінились). Фрагмент виводу для пристрою з трьома каналами (exp(1,0) – надходження, exp(2,5) – обслуговування, maxqueue = 3):

```
Create: exp(1.0), PROCESS: exp(2.5), 3 канали, maxqueue = 3

It's time for event in CREATE, time = 0.0000
CREATE state = 0 quantity = 1 tnext = 0.4734
PROCESS state = 1 queue = 0 quantity = 0 failure = 0 tnext = 0.7467
DISPOSE state = 0 quantity = 0 tnext = inf

It's time for event in CREATE, time = 0.4734
CREATE state = 0 quantity = 2 tnext = 0.7025
PROCESS state = 2 queue = 0 quantity = 0 failure = 0 tnext = 0.6215
DISPOSE state = 0 quantity = 0 tnext = inf

It's time for event in PROCESS, time = 0.6215
CREATE state = 0 quantity = 2 tnext = 0.7025
PROCESS state = 1 queue = 0 quantity = 1 failure = 0 tnext = 0.7467
DISPOSE state = 0 quantity = 1 tnext = inf

...

-------------RESULTS-------------
CREATE quantity = 914
PROCESS quantity = 837
    failure = 74, failure probability = 0.0810
    mean length of queue = 0.5486
    mean load = 0.6928
DISPOSE quantity = 837
created = 914, disposed = 837, failures = 74, in system at the end = 3
failure probability = 0.0810, mean number in system = 2.6270, mean time in system = 3.1386
```

Для перевірки модель порівнювалась із системою M/M/c/K за різної кількості каналів c (20 прогонів по 50000 одиниць часу).

Таблиця 4

Багатоканальний пристрій: імітація та теорія M/M/c/K ($\lambda = 1{,}0$, $\tau = 2{,}5$)

| c | maxqueue | $P_{відм}$, імітація | $P_{відм}$, теорія | Завантаження, імітація | Завантаження, теорія | Черга, імітація | Черга, теорія |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 0,6059 ± 0,0013 | 0,6062 | 0,9846 ± 0,0003 | 0,9845 | 2,3980 ± 0,0040 | 2,4006 |
| 2 | 3 | 0,2836 ± 0,0019 | 0,2837 | 0,8961 ± 0,0009 | 0,8954 | 1,4880 ± 0,0050 | 1,4863 |
| 3 | 3 | 0,1020 ± 0,0014 | 0,1024 | 0,7492 ± 0,0014 | 0,7480 | 0,7026 ± 0,0060 | 0,7005 |
| 4 | 3 | 0,0307 ± 0,0007 | 0,0308 | 0,6070 ± 0,0017 | 0,6058 | 0,2706 ± 0,0031 | 0,2697 |
| 5 | 3 | 0,0080 ± 0,0003 | 0,0082 | 0,4947 ± 0,0013 | 0,4959 | 0,0884 ± 0,0015 | 0,0904 |
| 3 | ∞ | 0,0000 ± 0,0000 | 0,0000 | 0,8320 ± 0,0020 | 0,8333 | 3,3890 ± 0,0752 | 3,5112 |
| 4 | ∞ | 0,0000 ± 0,0000 | 0,0000 | 0,6249 ± 0,0021 | 0,6250 | 0,5376 ± 0,0129 | 0,5331 |

Зі збільшенням кількості каналів ймовірність відмови та черга швидко зменшуються, а завантаження одного каналу падає пропорційно 1/c (табл. 4). Майже всі значення збігаються з теоретичними в межах довірчого інтервалу. Невеликі відхилення для c = 3 з необмеженою чергою (завантаження та черга) і для c = 5 (завантаження) пояснюються вхідним потоком: у цих 20 прогонах фактична інтенсивність надходжень $\hat{\lambda} = 0{,}9975$ замість 1. При повторенні з 40 іншими зернами отримано для c = 3 завантаження 0,8329 ± 0,0012 (теорія 0,8333) і чергу 3,458 ± 0,055 (теорія 3,511), для c = 5 – завантаження 0,4962 ± 0,0010 (теорія 0,4959).

Завдання 6:

Для виходу в кілька наступних блоків у класі Element замість одного посилання `nextElement` зберігається список маршрутів `routes` – пар «елемент, ймовірність переходу». Метод `add_route` додає маршрут, а `next_element()` обирає його методом оберненої функції для дискретної випадкової величини: генерується рівномірне число z і вибирається маршрут, у проміжок накопиченої ймовірності якого воно потрапило. Маршрут може вести до будь-якого елемента, зокрема до попереднього, тож у моделі можливі цикли. Для контролю кожен елемент рахує кількість вимог, відправлених кожним маршрутом (`route_counts`). Метод `set_next_element` залишився для одного виходу.

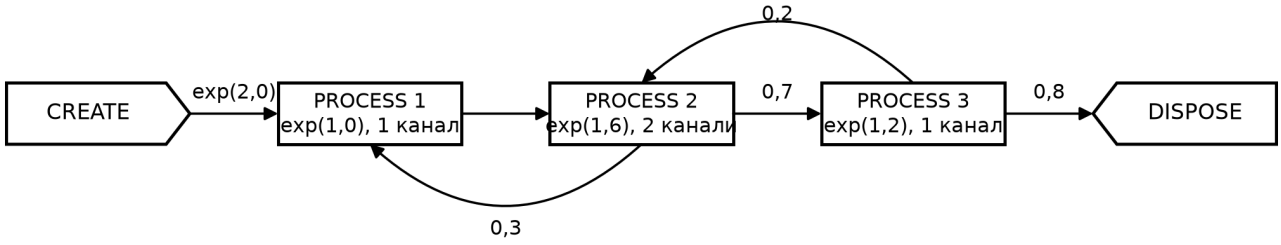



Рисунок 3 – Схема моделі з розгалуженням і поверненням у попередні блоки

Для перевірки побудовано модель за рисунком 3: після PROCESS 2 (два канали) вимога з ймовірністю 0,7 переходить у PROCESS 3, а з ймовірністю 0,3 повертається в PROCESS 1; після PROCESS 3 вона з ймовірністю 0,8 виходить із моделі, а з ймовірністю 0,2 повертається в PROCESS 2. Фрагмент виводу:

```
CREATE exp(2.0) -> PROCESS 1 exp(1.0) -> PROCESS 2 exp(1.6), 2 канали -> {PROCESS 3: 0.7; PROCESS
1: 0.3}; PROCESS 3 exp(1.2) -> {DISPOSE: 0.8; PROCESS 2: 0.2}

It's time for event in CREATE, time = 0.0000
CREATE state = 0 quantity = 1 tnext = 0.4630
PROCESS 1 state = 1 queue = 0 quantity = 0 failure = 0 tnext = 0.1961
PROCESS 2 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
PROCESS 3 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
DISPOSE state = 0 quantity = 0 tnext = inf

It's time for event in PROCESS 1, time = 0.1961
CREATE state = 0 quantity = 1 tnext = 0.4630
PROCESS 1 state = 0 queue = 0 quantity = 1 failure = 0 tnext = inf
PROCESS 2 state = 1 queue = 0 quantity = 0 failure = 0 tnext = 1.3537
PROCESS 3 state = 0 queue = 0 quantity = 0 failure = 0 tnext = inf
DISPOSE state = 0 quantity = 0 tnext = inf

...

-------------RESULTS-------------
CREATE quantity = 508
PROCESS 1 quantity = 770
    failure = 0, failure probability = 0.0000
    mean length of queue = 4.7251
    mean load = 0.7864
PROCESS 2 quantity = 873
    failure = 0, failure probability = 0.0000
    mean length of queue = 2.1275
    mean load = 0.7245
PROCESS 3 quantity = 598
    failure = 0, failure probability = 0.0000
    mean length of queue = 1.7062
    mean load = 0.6969
DISPOSE quantity = 480
created = 508, disposed = 480, failures = 0, in system at the end = 28
failure probability = 0.0000, mean number in system = 11.4911, mean time in system = 23.9397
Фактичні частки переходів за маршрутами:
  PROCESS 2: PROCESS 3: 0.695 (задано 0.7), PROCESS 1: 0.305 (задано 0.3)
  PROCESS 3: DISPOSE: 0.803 (задано 0.8), PROCESS 2: 0.197 (задано 0.2)
```

Фактичні частки переходів (PROCESS 2: 0,695 / 0,305, PROCESS 3: 0,803 / 0,197) відповідають заданим ймовірностям.

За необмежених черг і експоненційних затримок така модель є відкритою мережею Джексона, і кожен пристрій поводиться як система M/M/c з інтенсивністю надходжень $\lambda_i$, що визначається рівняннями балансу потоків:

$$\lambda_1 = \lambda_0 + 0{,}3\cdot\lambda_2,\quad \lambda_2 = \lambda_1 + 0{,}2\cdot\lambda_3,\quad \lambda_3 = 0{,}7\cdot\lambda_2,\quad \lambda_0 = 0{,}5.$$

Звідси $\lambda_1 = 0{,}7679$, $\lambda_2 = 0{,}8929$, $\lambda_3 = 0{,}6250$, тобто кожна вимога в середньому проходить PROCESS 1 1,536 раза, PROCESS 2 – 1,786 раза, PROCESS 3 – 1,25 раза. Середній час перебування в мережі за формулою Літтла дорівнює

сумі середніх кількостей вимог у пристроях, поділеній на $\lambda_0$. Результати 20 прогонів по 50000 одиниць часу:

Таблиця 5

Модель з поверненнями: імітація та теорія мереж Джексона

| **Елемент** | **Показник** | **Імітація (95 % ДІ)** | **Теорія** |
|---|---|---|---|
| PROCESS 1 | завантаження | 0,7674 ± 0,0032 | 0,7679 |
| PROCESS 1 | середня черга | 2,5308 ± 0,0725 | 2,5398 |
| PROCESS 1 | надходжень на одну вимогу | 1,5354 | 1,5357 |
| PROCESS 2 | завантаження | 0,7156 ± 0,0035 | 0,7143 |
| PROCESS 2 | середня черга | 1,5023 ± 0,0431 | 1,4881 |
| PROCESS 2 | надходжень на одну вимогу | 1,7841 | 1,7857 |
| PROCESS 3 | завантаження | 0,7513 ± 0,0035 | 0,7500 |
| PROCESS 3 | середня черга | 2,2638 ± 0,0720 | 2,2500 |
| PROCESS 3 | надходжень на одну вимогу | 1,2487 | 1,2500 |
| модель | час перебування | 18,4771 ± 0,2907 | 18,4487 |

Усі теоретичні значення лежать у довірчих інтервалах (табл. 5), отже розгалуження та повернення вимог реалізовано правильно. Модифікований клас працює й з обмеженими чергами: за maxqueue = 2 у кожному пристрої частка втрачених вимог становить 0,288, а середній час перебування зменшується до 9,22, бо вимоги, що мали б чекати в довгих чергах, отримують відмову.

Висновок до практикуму:

У роботі реалізовано об’єктно-орієнтований алгоритм імітації дискретно-подійних систем: модель складається з елементів – нащадків класу Element (Create, Process, Dispose), а клас Model просуває модельний час за принципом найближчої події. Така структура дозволяє будувати моделі довільної конфігурації простим з’єднанням елементів, не змінюючи алгоритму імітації.

Обчислення середнього завантаження накопиченням часу зайнятості пристрою дало значення, що збігаються з теоретичними для системи M/M/1/K. Модель за схемою з трьома послідовними пристроями показала очікувану поведінку: найбільше черг і відмов виникає на пристрої з найбільшим часом обслуговування.

Верифікація зміною 11 наборів вхідних змінних і параметрів показала логічно узгоджену реакцію моделі на кожну зміну та нульовий баланс вимог, а за сталих затримок модель дає точні значення завантаження й часу перебування.

Порівняння з аналітичними розв’язками для M/M/1, M/M/1/K, M/M/c/K і мережі Джексона показало збіг у межах 95 % довірчих інтервалів.

Клас Process модифіковано для обслуговування кількома ідентичними каналами та для виходу в кілька наступних блоків із заданими ймовірностями, зокрема з поверненням у попередні блоки. Обидві модифікації сумісні з попередніми моделями і підтверджені порівнянням з теорією масового обслуговування.